

EXPERIMENT 04 – JAVASCRIPT ALERTS AND CONFIRMS

1. Module Name and Experiment Title

Module Name: Module 1 – Automation with Selenium

Experiment Number: 04

Experiment Title: JavaScript Alerts and Confirms

Programming Language: Python

Website: DemoQA – Alerts

Website URL: <https://demoqa.com/alerts>

2. Problem Statement

To automate JavaScript alerts and confirmation dialogs using Selenium WebDriver with Python.

The experiment uses the DemoQA Alerts website. The automation script must trigger a JavaScript alert, retrieve its message, and accept it. It must also handle a JavaScript confirmation dialog by accepting and dismissing it. The results of these actions must be verified, and screenshots must be captured to document the execution.

3. Objective

The main objectives of this experiment are:

1. To understand how JavaScript alerts and confirmation dialogs work.
2. To automate JavaScript alerts using Selenium WebDriver.
3. To identify and interact with web elements using appropriate Selenium locators.
4. To switch control from the browser page to a JavaScript alert.
5. To retrieve and verify alert messages using Selenium.
6. To accept JavaScript alerts using the `accept()` method.
7. To handle confirmation dialogs using the `accept()` and `dismiss()` methods.
8. To verify the results of accepting and dismissing confirmation dialogs.
9. To use explicit waits for handling JavaScript dialogs.
10. To capture screenshots of the application and execution results.

4. Tools, Software and Concepts Used

Programming Language: Python

Automation Framework: Selenium WebDriver

Browser: Google Chrome

IDE: Visual Studio Code

Website: DemoQA – Alerts

Python Libraries:

- selenium
- time
- os

Selenium Modules:

- webdriver
- By
- WebDriverWait
- expected_conditions

Concepts Used:

- WebDriver initialization
- Browser maximization
- Element identification using ID
- JavaScript alert handling
- JavaScript confirmation dialog handling
- switch_to.alert
- accept() and dismiss() methods
- Explicit waits
- time.sleep() for observation
- Assertions
- Screenshot capture
- Exception-safe browser closure using finally

5. Source Code / Implementation Steps

Source Code

```
import os
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Create screenshot folder
os.makedirs("screenshots", exist_ok=True)

# Initialize browser
driver = webdriver.Chrome()
wait = WebDriverWait(driver, 15)

def pause():
    time.sleep(3)

def take_screenshot(filename):
    path = os.path.join("screenshots", filename)
```

```

driver.save_screenshot(path)
print("Screenshot saved:", path)

try:
    # Step 1: Open website
    driver.maximize_window()
    driver.get("https://demoqa.com/alerts")
    pause()

    # Scroll to the alert buttons
    alert_button = wait.until(
        EC.presence_of_element_located((By.ID, "alertButton"))
    )
    driver.execute_script(
        "arguments[0].scrollIntoView({block: 'center'});",
        alert_button
    )
    pause()

    take_screenshot("01_initial_page.png")

    # Step 2: Handle JavaScript Alert
    alert_button.click()
    alert = wait.until(EC.alert_is_present())

    print("Alert message:", alert.text)
    print("JavaScript alert is displayed.")

    # Pause to observe the alert
    pause()

    # Accept the alert
    alert.accept()
    pause()

    print("JavaScript Alert handled successfully!")

    # Step 3: Handle JavaScript Confirm - Accept
    confirm_button = wait.until(
        EC.element_to_be_clickable((By.ID, "confirmButton"))
    )
    confirm_button.click()

    confirm = wait.until(EC.alert_is_present())
    print("Confirm message:", confirm.text)

    # Pause to observe the confirmation dialog
    pause()

    confirm.accept()
    pause()

    result = wait.until(
        EC.visibility_of_element_located((By.ID, "confirmResult"))
    )

    assert result.text == "You selected Ok"
    print("JavaScript Confirm accepted successfully!")

    take_screenshot("02_confirm_accepted.png")

```

```

# Step 4: Handle JavaScript Confirm - Dismiss
confirm_button.click()
confirm = wait.until(EC.alert_is_present())

print("Confirmation dialog displayed again.")
pause()

confirm.dismiss()
pause()

result = wait.until(
    EC.visibility_of_element_located((By.ID, "confirmResult"))
)

assert result.text == "You selected Cancel"
print("JavaScript Confirm dismissed successfully!")

take_screenshot("03_confirm_dismissed.png")

print("All tasks completed successfully!")

# Keep browser open for observation
pause()

finally:
    driver.quit()

```

Implementation Steps

Step 1: Import Required Libraries

Import Selenium WebDriver, the `By` class, `WebDriverWait`, `expected_conditions`, the `time` module, and the `os` module.

Step 2: Initialize the WebDriver

Create a Chrome WebDriver instance, maximize the browser window, and initialize an explicit wait of 15 seconds.

Step 3: Open the Website

Navigate to the DemoQA Alerts website using the `driver.get()` method. Wait for three seconds to observe the page.

Step 4: Locate the Alert Button

Locate the JavaScript alert button using its ID, `alertButton`. Scroll the page to make the button visible and capture a screenshot of the initial page.

Step 5: Handle the JavaScript Alert

Click the alert button and wait for the JavaScript alert to appear. Retrieve and print the alert message using the `text` property. Accept the alert using the `accept()` method.

Step 6: Handle the JavaScript Confirmation Dialog

Locate and click the confirmation button using its ID, `confirmButton`. Wait for the confirmation dialog to appear and retrieve its message. Accept the dialog using the `accept()` method.

Step 7: Verify the Accepted Confirmation

Wait for the result element to become visible. Use an assertion to verify that the result is `You selected Ok`. Capture a screenshot of the accepted confirmation result.

Step 8: Dismiss the Confirmation Dialog

Click the confirmation button again and wait for the dialog to appear. Use the `dismiss()` method to cancel the dialog.

Step 9: Verify the Dismissed Confirmation

Wait for the result element to become visible and verify that it displays `You selected Cancel`. Capture a screenshot of the dismissed confirmation result.

Step 10: Close the Browser

Print the successful execution message and keep the browser open briefly for observation. Use a `finally` block to ensure that the browser is closed after execution.

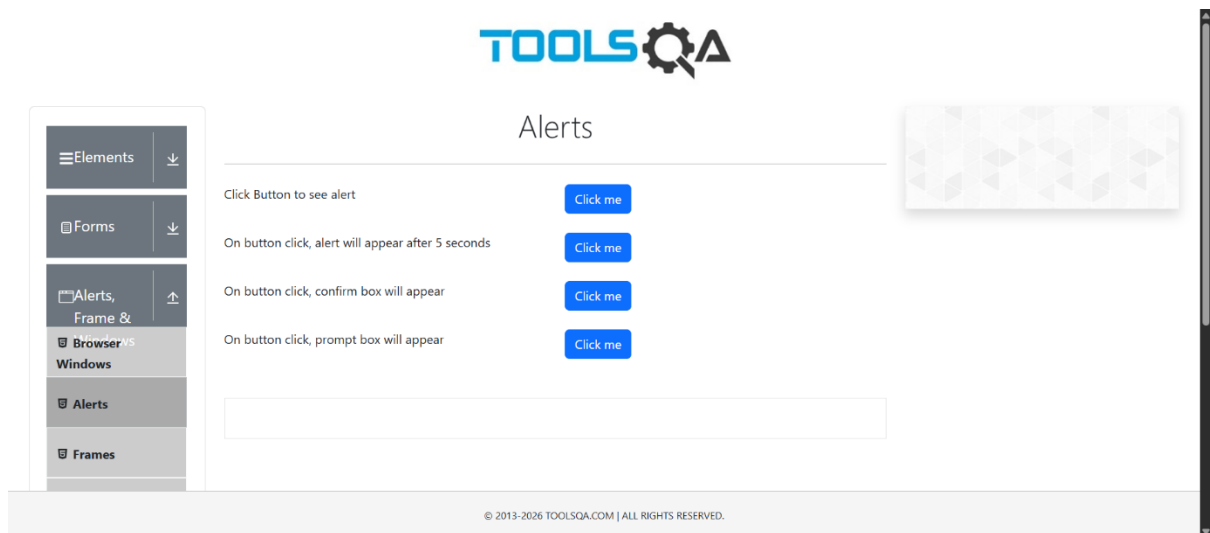
6. Output with Clearly Labelled Screenshots

The Selenium automation script was executed successfully on the DemoQA Alerts website.

The following screenshots demonstrate the initial state, JavaScript alert, confirmation dialog, confirmation results, and console output.

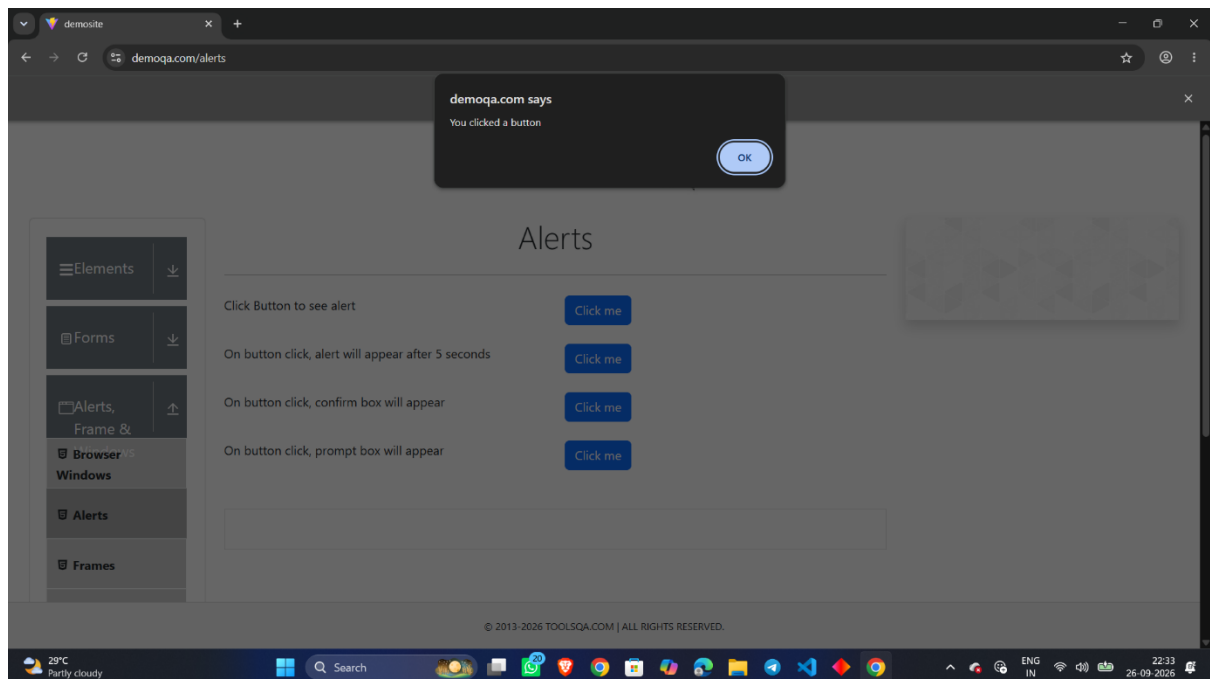
Screenshot 1: Initial DemoQA Alerts Web Page

Description: The DemoQA Alerts website is displayed before the automation begins. The page contains buttons for triggering JavaScript alerts and confirmation dialogs.



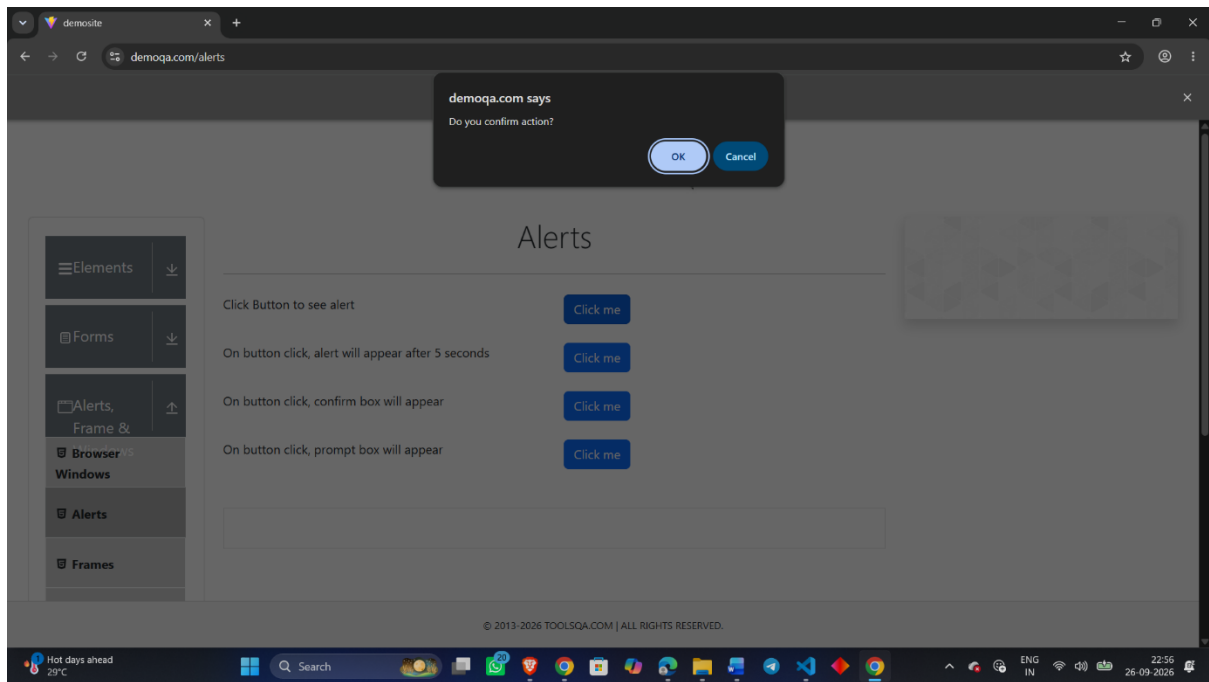
Screenshot 2: JavaScript Alert Dialog

Description: The browser displays a JavaScript alert containing the message **"You clicked a button"**. This screenshot demonstrates the alert before it is accepted by the Selenium automation script.



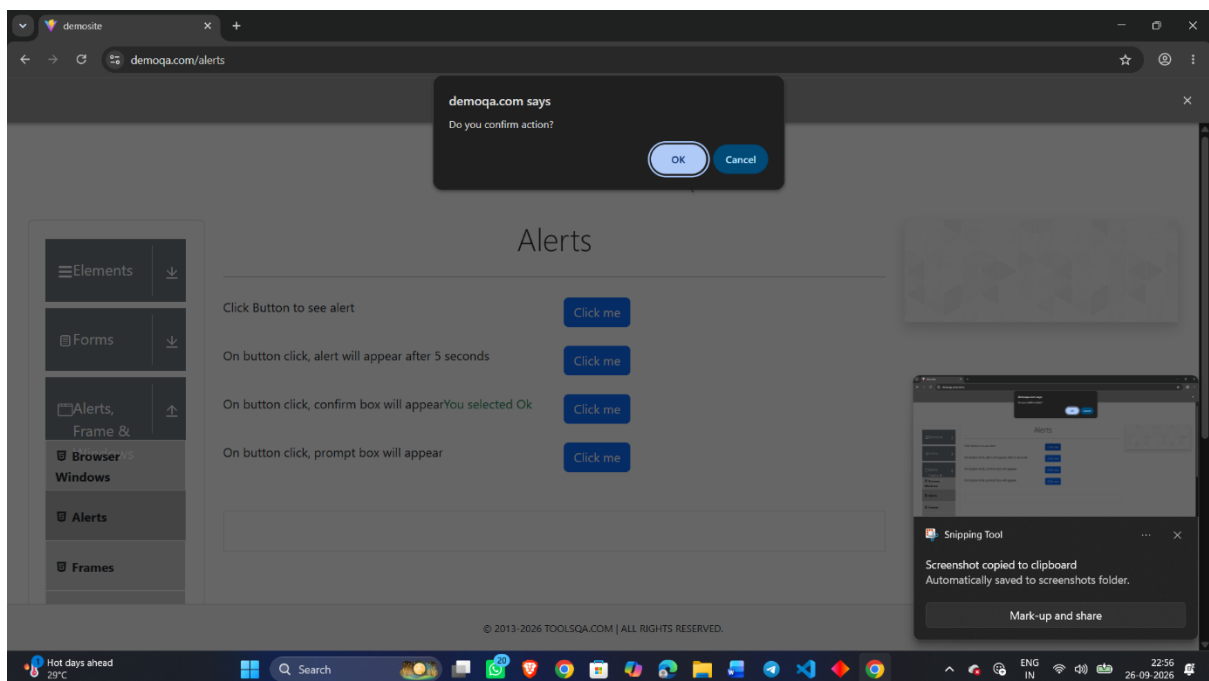
Screenshot 3: JavaScript Confirmation Dialog

Description: The browser displays a JavaScript confirmation dialog after clicking the confirmation button. The dialog contains the message **"Do you confirm action?"** with OK and Cancel options.



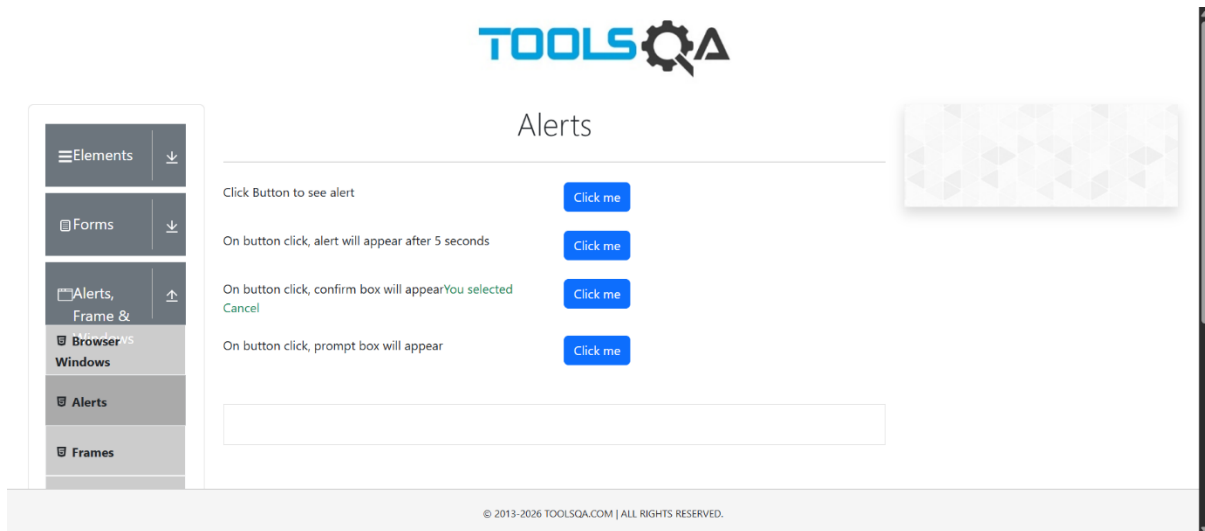
Screenshot 4: Confirmation Dialog Accepted

Description: The DemoQA Alerts page displays the result **"You selected Ok"** after the confirmation dialog has been accepted successfully.



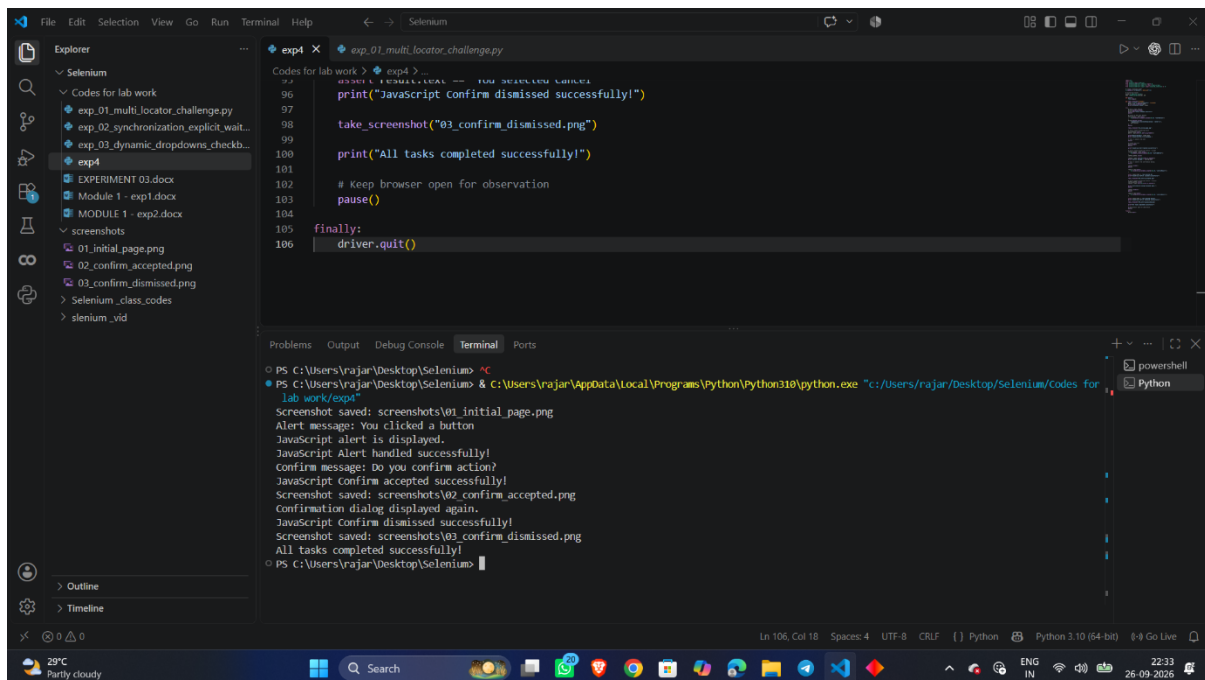
Screenshot 5: Confirmation Dialog Dismissed

Description: The DemoQA Alerts page displays the result **"You selected Cancel"** after the confirmation dialog has been dismissed successfully.



Screenshot 6: Successful Execution and Console Output

Description: The VS Code terminal displays the execution results. The output confirms that the JavaScript alert was handled successfully, the confirmation dialog was accepted and dismissed, and all tasks were completed.



Console Output:

```

Screenshot saved: screenshots\01_initial_page.png
Alert message: You clicked a button
JavaScript alert is displayed.
JavaScript Alert handled successfully!
Confirm message: Do you confirm action?
JavaScript Confirm accepted successfully!

```

Screenshot saved: screenshots\02_confirm_accepted.png
Confirmation dialog displayed again.
JavaScript Confirm dismissed successfully!
Screenshot saved: screenshots\03_confirm_dismissed.png
All tasks completed successfully!

7. Result, Observation and Conclusion

Result

The experiment was successfully executed using Selenium WebDriver with Python on the DemoQA Alerts website.

The automation script successfully performed the following tasks:

1. Opened the DemoQA Alerts website.
2. Located the JavaScript alert button.
3. Triggered the JavaScript alert.
4. Retrieved and displayed the alert message.
5. Accepted the JavaScript alert.
6. Triggered the JavaScript confirmation dialog.
7. Retrieved and displayed the confirmation message.
8. Accepted the confirmation dialog and verified the result.
9. Dismissed the confirmation dialog and verified the result.
10. Captured screenshots and displayed the successful execution message in the terminal.

Observation

- Selenium WebDriver successfully interacted with JavaScript alert and confirmation dialogs.
- The `switch_to.alert` interface was used to access the JavaScript dialogs.
- The `accept()` method successfully accepted the alert and confirmation dialog.
- The `dismiss()` method successfully cancelled the confirmation dialog.
- Explicit waits helped the script wait for the dialogs and page elements.
- Three-second pauses made it easier to observe the browser during execution.
- Assertions verified the results of accepting and dismissing the confirmation dialog.
- The script completed successfully and displayed the expected console output.

Conclusion

This experiment demonstrated how Selenium WebDriver can be used to automate JavaScript alerts and confirmation dialogs on a web page.

The experiment provided practical experience with alert handling, confirmation dialog acceptance and dismissal, explicit waits, assertions, and screenshot capture.

The successful execution of the script demonstrated the use of Selenium WebDriver for automating JavaScript dialogs and validating their behavior.